

Alte. Guillermo Brown



E.T. N°36 - D.E.15

Frascador Santino

5to 1ra

Turno Tarde

Escuela técnica et 36 almirante brown

Profesor: Oscar A Obregon

trabajo práctico escáner de red

Documentación

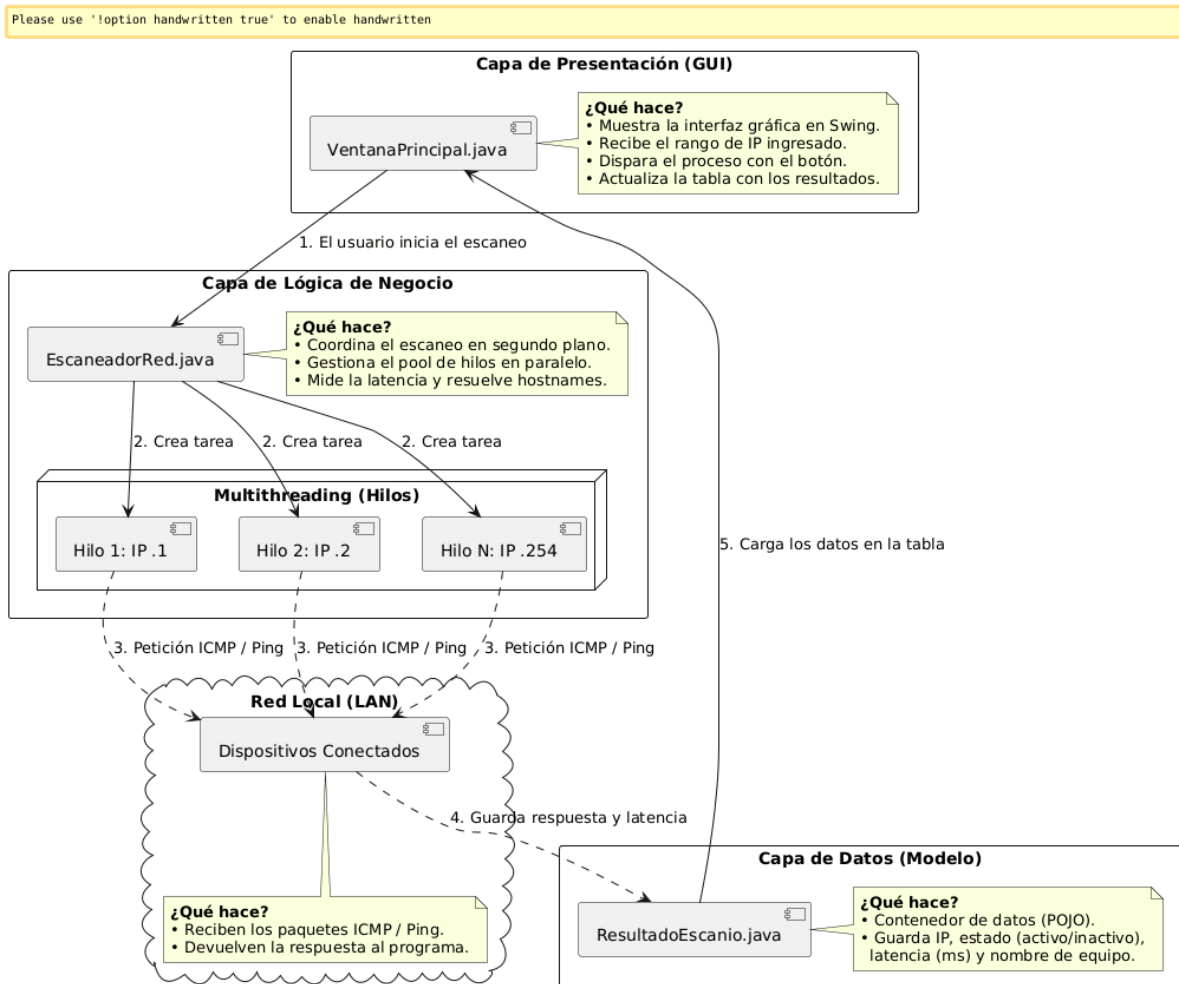
Para qué sirve el programa:

El programa Escáner de Red es una aplicación desarrollada en Java con interfaz gráfica (Swing) diseñada para analizar, identificar y monitorear todos los dispositivos conectados a una red local (LAN).

Su función principal es recorrer un rango determinado de direcciones IP (por ejemplo, desde 192.168.1.1 hasta 192.168.1.255) enviando peticiones de eco para verificar qué equipos como computadoras, teléfonos, impresoras o servidores se encuentran activos en ese momento. Además de detectar la presencia de cada dispositivo, la herramienta mide la

latencia o tiempo de respuesta en milisegundos, obtiene el nombre de host (*hostname*) correspondiente a cada dirección IP y organiza toda esta información en una tabla clara e interactiva para facilitar la auditoría y gestión del tráfico en la red.

Como esta armado el sistema:



El sistema opera mediante un flujo continuo entre sus tres capas principales para escanear la red sin congelar la interfaz:

1. Punto de partida (GUI): El usuario ingresa el rango de red en la clase `VentanaPrincipal.java` y presiona el botón para iniciar. La interfaz captura la orden y llama al backend.
2. Procesamiento en paralelo (Lógica): La clase `EscaneadorRed.java` toma el control y distribuye el trabajo creando múltiples hilos simultáneos (`Hilo 1`, `Hilo 2`, etc.). Cada hilo envía una petición de Ping (ICMP) a una dirección IP específica de la LAN para medir la latencia y consultar el nombre del equipo.
3. Estructuración y respuesta (Modelo): Los dispositivos conectados que responden devuelven sus datos a la lógica, la cual instancia objetos `ResultadoEscaneo.java` para empaquetar la IP, el estado activo, la latencia y el hostname.

4. Actualización visual: Finalmente, estos objetos cargan sus datos de vuelta en la tabla gráfica (`JTable`) de la interfaz, mostrando los resultados en tiempo real al usuario.

Qué métodos se usaron y por qué:

Métodos Principales Utilizados en el Código

Para construir el escáner de red distribuí las tareas en tres capas principales, aplicando los siguientes métodos en cada una:

1. En la Lógica de Negocio (EscaneadorRed.java)

- `Executors.newFixedThreadPool()`: Lo usé para manejar un pool de hilos en paralelo. De esta forma puedo escanear varias IP al mismo tiempo (por ejemplo, 50 a la vez) en lugar de probarlas de a una, lo que haría que el proceso tardara demasiado.
- `InetAddress.getByName()`: Convierte el texto con la IP que ingresa el usuario en un objeto de red que Java entiende para poder trabajar sobre él.
- `address.isReachable()`: Es el método clave que envía el paquete de Ping a la IP con un tiempo límite (*timeout*). Devuelve `true` si el equipo respondió o `false` si no lo hizo.
- `address.getHostName()`: Consulta a la red para intentar traer el nombre del dispositivo (como el nombre de la PC o del router) en vez de quedarme solo con la IP.
- `System.currentTimeMillis()`: Tomo el tiempo exacto antes y después del ping para restar ambos valores y calcular la latencia en milisegundos.

2. En la Interfaz Gráfica (VentanaPrincipal.java)

- `SwingUtilities.invokeLater()`: Es fundamental para refrescar la pantalla desde los hilos secundarios sin que la ventana del programa se congelara o se tildara mientras escanea.
- `DefaultTableModel.addRow()`: Lo usé para ir metiendo cada dispositivo encontrado como una nueva fila en la tabla de la pantalla a medida que van respondiendo.
- `actionPerformed()`: Escucha el clic en el botón de la interfaz para validar los datos e iniciar el proceso de escaneo.

3. En la Capa de Datos (ResultadoEscaneo.java)

- **Getters y Setters:** Sirven para encapsular los atributos de cada equipo (IP, nombre, si está activo y latencia) y pasárselos de forma limpia desde la lógica hacia la tabla de la interfaz.

Por qué se eligieron estas tecnologías

- **Java (JDK):** Lo elegí como lenguaje principal por su portabilidad y porque incluye de forma nativa todo lo necesario para trabajar con redes (`java.net`) y programación concurrente (`java.util.concurrent`), evitando la necesidad de instalar dependencias externas.
- **Librería Nativa `java.net (InetAddress)`:** Permite interactuar directamente con el protocolo IP. Utilicé el método `isReachable()` porque hace la prueba de conectividad (Ping/ICMP) de manera nativa dentro del código, sin tener que invocar comandos externos del sistema operativo (como ejecutar un ping por consola CMD o Terminal).
- **Concurrencia con `ExecutorService (Pool de Hilos)`:** Probar 254 direcciones IP de a una llevaría demasiado tiempo por los *timeouts*. Usar un pool de hilos con `Executors.newFixedThreadPool()` me permitió procesar las IP en paralelo, aprovechando los núcleos del procesador y reduciendo el escaneo a solo unos pocos segundos.
- **Java Swing:** Es la librería gráfica nativa de Java. La elegí porque es liviana y no requiere configuraciones adicionales, lo que me permitió armar una interfaz clara (JFrame, JTable, botones) y centrarme en la lógica del backend.
- **Visual Studio Code (IDE):** Lo seleccioné como entorno de desarrollo por ser un editor muy liviano, rápido y con bajo consumo de memoria RAM. Con el paquete de extensiones para Java (*Extension Pack for Java*), pude depurar y ejecutar el proyecto con la misma solidez que un IDE tradicional como NetBeans o Eclipse.
- **Arquitectura en 3 Capas (MVC):** Separar el proyecto en Presentación (gui), Lógica (logica) y Modelo (modelo) mantiene el código organizado. Esto permite que, si en el futuro se quiere cambiar la interfaz gráfica, la lógica de red y los datos se mantengan intactos sin tocar el motor del programa.

Qué problemas aparecieron y cómo se solucionaron:

Como problema en sí se me presentó el tema de si hacerlo en Eclipse o en Visual Studio Code ya que me costaba o era muy engorroso vincular Github con Eclipse, además de que no me acostumbro ni me gusta Eclipse, también durante el desarrollo, implementé una

arquitectura en 3 capas. Para lograr un escaneo rápido y sin fallas, resolví los cuatro problemas técnicos principales: evité el congelamiento de la interfaz derivando el trabajo pesado a hilos secundarios y actualizando la tabla con `SwingUtilities.invokeLater()`; reduje el tiempo de escaneo procesando hasta 50 IPs en paralelo con `Executors.newFixedThreadPool()`; mitigué los falsos negativos por firewall ajustando el timeout de `isReachable()` a 1000 ms; y eliminé las demoras en las consultas DNS ejecutando `getHostName()` únicamente sobre las IP que confirmaron estar activas.

Que se podría mejorar en el futuro:

Para el futuro, el proyecto se podría mejorar agregando un escáner de puertos TCP (como el 80 para HTTP o 22 para SSH) para saber qué servicios corren en cada equipo activo, la obtención de la dirección MAC para identificar el fabricante del dispositivo (ej. Apple, Samsung), la exportación de resultados a archivos como PDF por ejemplo, y la posibilidad de escanear subredes personalizadas más allá de las redes /24 habituales.